
Modül 22 — Kubernetes

Ders Notları

Spring Boot Edu



2026-09-25

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 İstenen Durum	2
2.2 Request'ler ve Limit'ler	2
2.3 Kustomize	3
3. Adım Adım Örnekler	3
3.1 Image ve Deployment Script'i	3
3.2 Base: Yapılandırma ve Sırlar	4
3.3 Deployment	4
3.4 Overlay'ler: dev ve prod	7
3.5 StatefulSet Olarak PostgreSQL	8
3.6 Otomatik Ölçekleme, Rolling Update ve Rollback	9
4. Sık Yapılan Hatalar ve En İyi Pratikler	10
5. Özet	11
6. İleri Okuma	11

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- kind ile yerel bir Kubernetes kümesi çalıştırmak ve kendi image'ınızı ona yüklemek
- Bir uygulamayı Deployment, Service, ConfigMap ve Secret ile tarif etmek
- Liveness, readiness ve startup probe'larını Spring Boot Actuator'a bağlamak
- Kaynak request ve limit'lerini ayarlamak ve her birinin ne yaptığını açıklamak
- Kustomize ile ortam varyantları oluşturmak (base + overlay'ler)
- Bir HorizontalPodAutoscaler ile ölçeklemek, rolling update ile güncellemek ve bozuk bir sürümü geri almak

Ön koşullar: Modül 20 (image, probe'lar, zarif kapanış) · **Tahmini süre:** 5 saat · **Docker, kind ve kubectl gerekir**

2. Kavramlar

2.1 İstenen Durum

Kubernetes ile “iki container başlat” gibi komutlar çalıştırmazsınız. **İstenen durumu** YAML ile tarif edersiniz (“bu image’ın iki kopyası, bu probe’larla”) ve onu uygularsınız. Controller’lar istenen durumu sürekli gerçeklikle karşılaştırır ve harekete geçer: Çöken bir pod değiştirilir, yeni bir image sürümü adım adım yayılır.

Nesne	Amaç
Pod	birlikte çalışan bir veya daha fazla container; değiştirilebilir
Deployment	N özdeş pod’u çalışır tutar ve onları günceller
Service	tüm hazır pod’lar için tek bir sabit ad ve adres (yük dengeleme)
ConfigMap / Secret	ortam değişkenleri veya dosyalar olarak yapılandırma ve kimlik bilgileri
StatefulSet	sabit bir kimliği ve kendi depolaması olan pod’lar (veritabanları)
HorizontalPodAutoscaler	kopya sayısını yüke göre değiştirir

2.2 Request’ler ve Limit’ler

	Anlamı	Etkisi
requests	pod'un kesin aldığı	scheduler pod'ları request'lere göre yerleştirir; HPA yüzdelerini onlardan hesaplar
limits	üst sınır	limitin üstünde bellek: container öldürülür (OOMKilled); limitin üstünde CPU: yavaşlatılır

Ders bir bellek limiti koyar ama CPU limiti koymaz. Bir CPU limiti JVM'i tam da CPU'ya ihtiyaç duyduğu anda (başlangıç, çöp toplama) yavaşlatır ve hiçbir şeyi korumaz.

2.3 Kustomize

Kustomize (kubectl'e yerleşik) bir **base**'i **overlay**'lerle birleştirir. Base tüm ortamların paylaştığı şeyleri içerir. Bir overlay yalnızca farklı olanı değiştirir (kopya sayısı, kaynaklar, yapılandırma). Şablon yok, değişken yok: düz YAML artı yamalar.

3. Adım Adım Örnekler

Kümeyi bir kez oluşturun. Script, otomatik ölçekleyicinin ihtiyaç duyduğu metrics-server'ı da kurar:

```
scripts/kind-up.sh
```

3.1 Image ve Deployment Script'i

Uygulama modül 20'dekidir (API, probe'lar, zarif kapanış), artı bir CPU yükü endpoint'i. Image aynı çok aşamalı Dockerfile ile build edilir. `deploy.sh` tüm adımları yapar:

```
modules/22-kubernetes/deploy.sh dev
```

1. `./mvnw package` ve `docker build`
2. `kind load docker-image`: `kind` node'unun kendi image deposu vardır ve Docker'ınızın image'larını göremez. Bu yüzden Deployment `imagePullPolicy: IfNotPresent` kullanır.
3. `kubectl apply -k modules/22-kubernetes/k8s/overlays/dev`

4. `kubectl rollout status`: Tüm pod'lar hazır olana kadar bekler
5. Service üzerinden bir smoke test (port-forward)

```
deployment "bookstore" successfully rolled out
NAME                                READY   STATUS    RESTARTS   AGE
bookstore-5b6b97d4d7-wdkxg         1/1     Running   0           4s
postgres-0                         1/1     Running   0           82s
{"host": "bookstore-5b6b97d4d7-wdkxg", "name": "Bookstore (dev)", "currency": "TRY"}
```

3.2 Base: Yapılandırma ve Sırlar

```
configMapGenerator:
  - name: bookstore-config           # the real name gets a hash suffix, e.g.
    ↪ bookstore-config-7k2m9f4b8c
    literals:
      - BOOKSTORE_SHOP_NAME=Bookstore on Kubernetes
      - BOOKSTORE_SHOP_CURRENCY=TRY
secretGenerator:
  - name: bookstore-db
    literals:
      - SPRING_DATASOURCE_USERNAME=bookstore
      - SPRING_DATASOURCE_PASSWORD=bookstore # a dev value only – a real secret comes
    ↪ from outside the repository
```

Bir **generator**, ConfigMap'i adında içeriğinin bir hash'iyle oluşturur (bookstore-config-tk8m75d8kc) ve Kustomize bu adı Deployment'a yazar. Bir değer değiştiğinde ad değişir. Deployment da değişir ve Kubernetes pod'ları yeniler (rolling). Elle yazılmış bir ConfigMap sessizce değişirdi: Ortam değişkenleri yalnızca bir container başladığında okunur.

! Uyarı

Bir Secret yalnızca base64 ile kodlanmıştır, şifrelenmemiştir. Generator'daki parola bir geliştirme değeridir. Gerçek bir sistemde sırlar deponun dışından gelir (bir secret deposu, Sealed Secrets, External Secrets).

3.3 Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bookstore
spec:
```

```
replicas: 2
selector:
  matchLabels:
    app: bookstore
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0 # never fewer ready pods than wanted during an
↪ update
    maxSurge: 1 # start one new pod at a time
template:
  metadata:
    labels:
      app: bookstore
  spec:
    terminationGracePeriodSeconds: 30 # longer than
↪ spring.lifecycle.timeout-per-shutdown-phase (20 s)
    initContainers: # runs to completion before the application
↪ starts
      - name: wait-for-postgres
        image: pgvector/pgvector:0.8.6-pgl8
        command: ["sh", "-c", "until pg_isready -h postgres -U bookstore -d bookstore;
↪ do sleep 2; done"]
    containers:
      - name: bookstore
        image: springbootedu/kubernetes:1.0.0
        imagePullPolicy: IfNotPresent # the image is loaded into kind, not pulled
↪ from a registry
        ports:
          - name: http
            containerPort: 8080
        envFrom:
          - configMapRef:
              name: bookstore-config
          - secretRef:
              name: bookstore-db
        env:
          - name: SPRING_DATASOURCE_URL
            value: jdbc:postgresql://postgres:5432/bookstore
        resources:
          requests: # what the scheduler reserves (and what the HPA
↪ percentages refer to)
            cpu: 250m
            memory: 384Mi
          limits:
            memory: 512Mi # above this the container is killed
↪ (OOMKilled); no CPU limit: no throttling
```

```

    startupProbe:                                # until it succeeds, the other probes wait
↪ (slow first start)
    httpGet:
      path: /actuator/health/liveness
      port: http
      periodSeconds: 2
      failureThreshold: 60                      # up to 2 minutes to start
    livenessProbe:                               # fails → the container is restarted
    httpGet:
      path: /actuator/health/liveness
      port: http
      periodSeconds: 10
      failureThreshold: 3
    readinessProbe:                             # fails → the pod gets no traffic from the Service
    httpGet:
      path: /actuator/health/readiness
      port: http
      periodSeconds: 5
      failureThreshold: 2

```

- **initContainer**, PostgreSQL cevap verene kadar bekler. O olmadan ilk başlangıç başarısız olur (Flyway bağlanamaz) ve Kubernetes veritabanı ayağa kalkana kadar pod'u birkaç kez yeniden başlatır.
- envFrom, ConfigMap'ın ve Secret'ın her anahtarını bir ortam değişkenine çevirir. Gerisini Boot'un relaxed binding'i yapar (modül 20).
- terminationGracePeriodSeconds: 30, Spring'in timeout-per-shutdown-phase: 20s'inden uzundur. Kubernetes SIGTERM gönderir, bekler ve ancak ondan sonra öldürür.

Probe'lar Actuator gruplarını gösterir:

```

startupProbe:                                # until it succeeds, the other probes wait (slow first
↪ start)
    httpGet:
      path: /actuator/health/liveness
      port: http
      periodSeconds: 2
      failureThreshold: 60                      # up to 2 minutes to start
    livenessProbe:                             # fails → the container is restarted
    httpGet:
      path: /actuator/health/liveness
      port: http
      periodSeconds: 10
      failureThreshold: 3
    readinessProbe:                             # fails → the pod gets no traffic from the Service
    httpGet:
      path: /actuator/health/readiness

```

```
port: http
periodSeconds: 5
failureThreshold: 2
```

Startup probe yavaş bir ilk başlangıcı korur: O başarılı olana kadar liveness çalışmaz, böylece 40 saniyeye ihtiyaç duyan bir JVM 30 saniye sonra yeniden başlatılmaz.

ManifestPolicyTest bu tür kuralları her build’de, küme olmadan kontrol eder. Örneğin Kubernetes, uygulamanın zarif kapanışından daha uzun beklemelidir:

```
@Test
void kubernetesWaitsLongerThanTheGracefulShutdown() throws IOException {
    int gracePeriod = (Integer) path(deployment(), "spec", "template", "spec",
        ↪ "terminationGracePeriodSeconds");
    String shutdownTimeout =
        ↪ Files.readString(Path.of("src/main/resources/application.yaml"))
        .lines().filter(line ->
            ↪ line.contains("timeout-per-shutdown-phase")).findFirst().orElseThrow();
    int springSeconds = Integer.parseInt(shutdownTimeout.replaceAll(".*: (\\d+)s.*",
        ↪ "$1"));

    assertThat(gracePeriod).isGreaterThan(springSeconds);
}
```

3.4 Overlay’ler: dev ve prod

```
k8s/
  base/           namespace, postgres, deployment, service, generators
  overlays/dev/    1 replica, shop name "Bookstore (dev)"
  overlays/prod/   more memory, HorizontalPodAutoscaler 2-5 replicas
```

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: bookstore                                # also for the resources this overlay adds
↪ (hpa.yaml)
resources:
  - ../../base
  - hpa.yaml
patches:
  - path: resources.yaml
configMapGenerator:
  - name: bookstore-config
    behavior: merge
```

literals:

- BOOKSTORE_SHOP_NAME=Bookstore

kubectl kustomize modules/22-kubernetes/k8s/overlays/prod sonucu uygulamadan yazdırır.

!! Dikkat

Base'deki namespace: bookstore yalnızca base'in kaynaklarına uygulanır. Bir overlay'in eklediği kaynak (burada hpa.yaml) default namespace'ine düşer ve orada hiçbir Deployment bulamaz. Bu yüzden her overlay de namespace: ayarlar ve ManifestPolicyTest bunu kontrol eder. (Bu, bu ders yazılırken oldu.)

3.5 StatefulSet Olarak PostgreSQL

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: pgvector/pgvector:0.8.6-pg18
          env:
            - name: POSTGRES_DB
              value: bookstore
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: bookstore-db
                  key: SPRING_DATASOURCE_USERNAME
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: bookstore-db
```

```

        key: SPRING_DATASOURCE_PASSWORD
      - name: PGDATA
        value: /var/lib/postgresql/data/pgdata
    ports:
      - containerPort: 5432
    readinessProbe:
      exec:
        command: ["pg_isready", "-U", "bookstore", "-d", "bookstore"]
      periodSeconds: 5
    volumeMounts:
      - name: data
        mountPath: /var/lib/postgresql/data
    volumeClaimTemplates:
      # every pod of the StatefulSet gets its own
      ↪ persistent volume
      - metadata:
          name: data
        spec:
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 1Gi

```

Bir StatefulSet her pod'a sabit bir ad (postgres-0) ve volumeClaimTemplates'ten kendi volume'ünü verir. Volume pod'un yeniden başlamasından sonra da kalır. Bu öğrenmek için yeterlidir. Production'da yönetilen bir veritabanı veya bir operator kullanın (yedekler, replikasyon, yükseltmeler).

3.6 Otomatik Ölçekleme, Rolling Update ve Rollback

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: bookstore
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: bookstore
  minReplicas: 2
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu

```

```
target:
  type: Utilization
  averageUtilization: 60
```

İkinci bir pod'dan gelen yükü (/api/work CPU'yu meşgul tutar) otomatik ölçekleyici bir dakika içinde tepki verdi:

```
modules/22-kubernetes/deploy.sh prod
kubectl -n bookstore run load --image=busybox:1.37 --restart=Never -- /bin/sh -c \
  'while true; do for i in 1 2 3 4 5 6 7 8; do wget -q -O-
    ↪ "http://bookstore/api/work?millis=200" > /dev/null & done; wait; done'
kubectl -n bookstore get hpa bookstore --watch          # replicas 2 → 4 → 5
kubectl -n bookstore delete pod load
```

Yük durduktan sonra HPA pod'ları tekrar kaldırmadan önce 5 dakika bekler (varsayılan stabilizasyon penceresi).

Bozuk bir sürüm ve bir rollback:

```
kubectl -n bookstore set image deployment/bookstore
↪ bookstore=springbootedu/kubernetes:1.0.1-broken
kubectl -n bookstore get pods          # the new pod: ErrImagePull – the 5 old pods
↪ still serve
kubectl -n bookstore rollout history deployment/bookstore
kubectl -n bookstore rollout undo deployment/bookstore
```

maxUnavailable: 0 sayesinde Kubernetes eski bir pod'u ancak yenisi hazır olduğunda kaldırır. Hiç hazır olmayan bir sürüm (yanlış image, başarısız readiness) bu yüzden trafiği asla elinden almaz. Rollout, siz geri alana kadar yalnızca durur.

İşiniz bittiğinde kümeyi kaldırın: scripts/kind-down.sh.

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! Dikkat

Liveness, veritabanına veya başka servislere bağlı olmamalıdır (modül 20). Kubernetes'te başarısız bir liveness probe'u pod'u yeniden başlatır ve tüm pod'ların bir yeniden başlatma döngüsü, bir veritabanı hıçkırığını bir kesintiye çevirir.

- **Yapın:** Bellek request ve limit'lerini ve CPU request'lerini ayarlayın. CPU request'leri olmadan HPA bir yüzde hesaplayamaz.
- **Yapmayın:** Image etiketi olarak latest kullanmayın. Kubernetes sürümleri ayırt edemez ve bir rollback'in dönecek yeri olmaz.

- **Yapın:** JVM'e başlaması için zaman verin (startup probe) ve zarif kapanışa bitmesi için zaman verin (terminationGracePeriodSeconds).
- **Yapmayın:** Elle yazılmış bir ConfigMap'i değiştirip çalışan pod'ların onu görmesini beklemeyin. Bir generator kullanın veya Deployment'ı yeniden başlatın (kubectl rollout restart).
- **Yapın:** Manifest'leri yalnızca kümede değil build'de de kontrol edin (bir policy testi, kubectl kustomize veya kubeconform gibi araçlar).

5. Özet

- Kubernetes istenen durumla çalışır: Deployment'lar pod'ları çalışır tutar, Service'ler onlara tek bir adres verir.
- Probe'lar: İlk başlangıç için startup, "beni yeniden başlat" için liveness, "bana trafik gönder" için readiness. Üçü de Actuator'dan gelir.
- Request'ler zamanlama ve otomatik ölçekleme içindir, limit'ler üst sınırlardır. Bellek limiti evet, CPU limiti pek değil.
- Kustomize ortamları tek bir base'den oluşturur. Generator'lar yapılandırma değişikliklerini rolling update'lere çevirir.
- HPA CPU'ya göre ölçekler. maxUnavailable: 0 artı readiness bir rolling update'i güvenli yapar ve rollout undo geri döner.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Boot — Deploying to Kubernetes](#)
- [Kubernetes — Deployments · Probes](#)
- [Kubernetes — Resource management · HorizontalPodAutoscaler](#)
- [Kustomize · kind](#)